

Guion — Video 2 Wireshark: hallazgos de la captura

(Lab 02, sección 3.4)

Grupo: Camilo Aguirre (CA) y Juan David Rangel (JD) — Curso AYSR Duración objetivo: ~7 minutos | Herramienta: Wireshark Reparto 50/50 en 2 tomas: **Mitad 1 (0:00–3:30): JD** | **Mitad 2 (3:30–7:00): CA** Temas: captura real de scielo.org.co, encapsulación capa por capa (Frame → Ethernet II → IPv4 → TCP → HTTP), diferencias entre paquetes. Base técnica: datos reales de lab-evidencias/red/analisis-red-completo.txt (captura wireshark-scielo.pcapng, 2742 paquetes en 30 s). La parte de “3 recursos web” NO aplica (grupo de 2). Guion: texto para LEER EN VOZ ALTA (natural), no para copiar textual en ningún informe. Cada mitad es UN SOLO TAKE continuo; las indicaciones de pantalla son notas de la toma, no cortes de edición.

Estructura y tiempos (total ~7:00)

Mitad	Quién	Horario	Temas cubiertos
Mitad 1 — un solo take	JD	0:00–3:30	Presentación del grupo; la captura real (2742 paquetes, modo promiscuo, tráfico mixto); hacia el GET (DNS, handshake TCP, frame 395, HTTP 200, RTT); diferencias entre paquetes (HTTP claro vs TLS, QUIC, DNS, ARP, ICMP, tamaños, retransmisiones)
Mitad 2 — un solo take	CA	3:30–7:00	Handoff; encapsulación capa por capa (Frame → Ethernet II → IPv4 → TCP → HTTP, con MACs/IPs/puertos reales); salud y seguridad de la red; cierre de los hallazgos y despedida
TOTAL	JD + CA	0:00–7:00	Reparto 50/50: JD 3:30 / CA 3:30

Mitad 1 [JD] — 0:00–3:30 (un solo take)

(En pantalla: portada del video + la captura wireshark-scielo.pcapng abierta. Después la lista de paquetes completa y Statistics → Protocol Hierarchy; luego filtrar y localizar los frames 361 a 395 con el handshake y el GET; al final comparar paquetes HTTP, TLS, DNS, ARP y QUIC en la lista.)

JD: “Hola, somos Camilo Aguirre y Juan David Rangel. En este video les mostramos los hallazgos de la captura real que hicimos con Wireshark: nuestra visita a www.scielo.org.co. Vamos a ver qué contiene la captura, cómo viaja un paquete capa por capa —la encapsulación— y las diferencias entre los paquetes que aparecen.

La captura duró 30 segundos y registró **2.742 paquetes**, unos tres millones de bytes. La hicimos así: abrimos Wireshark, iniciamos la captura en la interfaz Wi-Fi y visitamos www.scielo.org.co **en HTTP, sin cifrar, a propósito**: así el tráfico se ve en claro. La captura no contiene solo nuestra visita: como la tarjeta está en modo promiscuo, registró todo el tráfico del segmento. Por eso aparecen más de 2.500 paquetes de streaming de Google de fondo, mientras que la conversación con SciELO fueron solo 36 paquetes: la visita propiamente dicha.

Veamos qué pasó antes de que llegara la página. Primero, **DNS**: dos paquetes, la consulta y la respuesta, para resolver www.scielo.org.co a la dirección IP **168.176.28.57**. Después, el **handshake TCP** de tres pasos: SYN, SYN-ACK y ACK, que vemos en los frames 361 a 394. Recién entonces, en el **frame 395**, viaja la petición **GET / HTTP/1.1**, con el encabezado Host: www.scielo.org.co. El servidor responde con HTTP 200 y 9.200 bytes de la página. La latencia promedio de la conexión, el RTT, fue de 150 milisegundos.

Y ahora las diferencias entre los paquetes, porque en la misma captura hay paquetes muy distintos. Los de SciELO van en **HTTP por el puerto 80, en texto plano**: se lee la petición GET completa. En cambio, el tráfico hacia Google y Cloudflare va **cifrado con TLS 1.3 por el 443**: solo se ven los paquetes del handshake y el contenido no se puede leer. Y Chrome también usó **QUIC**, el HTTP/3, que corre sobre UDP en el puerto 443. El **DNS** usa UDP 53 para resolver nombres, y el **ARP**, con solo 3 paquetes, resuelve direcciones MAC dentro de nuestra red local. Incluso apareció un único paquete ICMP, un ping. También difieren en tamaño: el streaming de Google generó más de 2.500 paquetes; la conversación con SciELO, 36. Y hubo 17 retransmisiones: paquetes que se perdieron en el Wi-Fi y se volvieron a enviar, algo normal en esta tecnología. Ahora, Camilo les explica la encapsulación del paquete GET, capa por capa.”

Mitad 2 [CA] — 3:30–7:00 (un solo take)

(En pantalla: el paquete GET expandido capa por capa, de abajo hacia arriba. Después la tabla resumen del análisis o las estadísticas de la captura; al final, portada.)

CA: “Y ahora el corazón del análisis: la encapsulación, capa por capa, de abajo hacia arriba. Si expandimos el paquete GET, la primera entrada es **Frame**: la trama completa, con su tamaño en el medio físico. Le sigue **Ethernet II**, la capa 2: aquí están las direcciones MAC, la del gateway ZTE, e0:a1:ce:d2:85:76, como destino, y la de nuestra tarjeta, 08:f9:7e:9e:25:03, como origen. Encima va **Internet Protocol**, la capa 3: IP origen 192.168.1.6 —nuestro PC— e IP destino 168.176.28.57 —SciELO—. Después **TCP**, la capa 4: un puerto origen efímero y el puerto destino 80, el de HTTP. Y arriba de todo, **HTTP**, la capa de aplicación, con la petición GET. Cada capa agrega su encabezado a los datos de la capa superior: eso es la encapsulación. En el servidor se hace el proceso inverso, la desencapsulación, en orden de arriba hacia abajo.

¿Qué dice la captura sobre la salud de la red? Las retransmisiones fueron el 0,6% del tráfico, con 14 ACK duplicados para recuperar los segmentos perdidos, y ninguna ventana TCP saturada: no hubo congestión. Conclusión: la red está sana. El Wi-Fi pierde algunos paquetes —es normal— y TCP los recupera garantizando orden y confiabilidad. En seguridad, la visita a SciELO fue en claro, y todo lo demás iba cifrado; en esos 30 segundos no vimos tráfico sospechoso ni indicios de envenenamiento ARP.

Tres hallazgos principales. Primero, la encapsulación se ve concreta en un paquete real: Frame, Ethernet, IP, TCP y HTTP, cada capa con su encabezado. Segundo, los paquetes no son todos iguales: HTTP en claro, TLS cifrado, UDP para DNS y QUIC, ARP para la red local, y cada protocolo cumple su función. Tercero, la red funciona bien: las retransmisiones existen y TCP las corrige. Estas ideas —protocolos, capas, encapsulación— son las del curso, y las vimos funcionando en vivo en Wireshark. Con esto cerramos los dos videos de Wireshark: la interfaz y los filtros en el primero, y los hallazgos de la captura real en este. El archivo de evidencia, wireshark-scielo.pcapng, queda guardado para el informe del laboratorio, junto con los screenshots del paquete GET. ¡Gracias por vernos!”

Notas

- **Reperto 50/50:** JD 3:30 (mitad 1) y CA 3:30 (mitad 2), cada uno en un solo take.
- **Datos usados (reales, de analisis-red-completo.txt):** 2742 paquetes / 3.004.210 bytes en 30 s; IP local 192.168.1.6; gateway ZTE e0:a1:ce:d2:85:76; NIC 08:f9:7e:9e:25:03; DNS 2 paquetes; handshake TCP frames 361–394; GET frame 395; respuesta HTTP 200 de 9.200 bytes; RTT promedio 150 ms; retransmisiones 17 (0,6%); 14 ACK duplicados; conversación SciELO 36 paquetes; streaming Google 2.582 paquetes; ARP 3 paquetes; ICMP 1.
- **Cuidado con los números:** WIRESHARK-LAB02.md registra una captura anterior distinta (1691 paquetes, frame 211, IP 192.168.1.10). Para el video usar SIEMPRE los números de analisis-red-completo.txt, que es la evidencia final.
- **Idioma:** español neutro, sin tecnicismos innecesarios; quedan bien las secciones donde se ve la herramienta mientras se explica.

Cómo grabar

- Grabar la **mitad 1 (JD)** y la **mitad 2 (CA)** en **UN SOLO TAKE** cada una, sin pausas ni cortes internos.
- Dejar **1–2 segundos de silencio** al inicio y al final de cada toma (facilita el recorte en la edición).
- Grabar ambas tomas con la **misma resolución y fps** (recomendado 1080p / 30 fps), preferentemente el mismo día y con la misma luz.
- Guardar como archivos separados: video3-mitad1.mp4 (JD) y video3-mitad2.mp4 (CA).
- Juntarlas después en ese orden siguiendo GUIA-EDICION-VIDEOS-LAB02.md.
- Duración máxima del video final: **7:00** (mitad 1 ≈ 3:30 + mitad 2 ≈ 3:30).